

COMP40009: C Project Final Report

Group 19

Ashlyn Chor

Katie Wan

Suyoung Moon

Jiahui Yan

June 2026

1 Overview

In this report, we will be discussing our implementation of the ARMv8 assembler, how we made an LED blink using a Raspberry Pi, our design and architecture for a vocal synthesiser, how we tested the project and a reflection of our group dynamics, including our individual experiences.

2 Assembler Implementation

2.1 Two Passes

We chose to use a two-pass for the assembler to easily handle forward references. This was chosen over using a single pass as it would require additional complexities to handle a label being referenced before they are defined. Compared to two-pass in which all label addresses are known before parsing.

2.2 Symbol Table (Pass 1)

In the first pass, the assembler reads each line of assembly and tokenises the line using `strtok_r`. A global variable, address, is maintained and incremented for every instruction or directive it encounters. Any labels encountered are added to a symbol table. A resizing array of pairs mapping label names to memory address is used for the symbol table. This is to account for the fact that an unknown number of labels may be used in the assembly file. We considered using a resizing hashmap for $O(1)$ lookup however we concluded it to be too complex to implement and unnecessary for the scope of the project.

2.3 Parsing and Encoding (Pass 2)

In the second pass, the assembler rescans the assembly code from the start of the file. If the line is a label or an empty line, then it is skipped. If not, then each instruction is parsed by its corresponding parse function into `Instruction_IR` struct. Any label referenced is looked up in

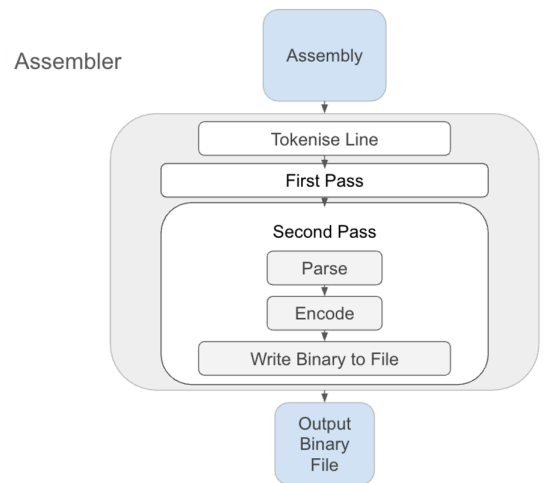


Figure 1: Assembler Structure

the symbol table. The `Instruction_IR` is then used to encode the instruction into binary and written to the output file. Any errors (undefined labels, invalid opcodes, etc.) are reported.

We chose to first parse from assembly to `Instruction_IR` before encoding it to binary, separating string handling from bit manipulation. The struct acts as an intermediate representation which can then be mechanically translated into bits binary form by applying the corresponding shifts and mask to the fields. This keeps the two stages independently testable and easier to debug.

3 Flash the LED (Part III)

3.1 Hardware Configuration

We built a simple circuit with an LED and a resistor. The LED is connected to GPIO 2 in Raspberry Pi.

3.2 Memory Configuration

Initially we used the `‘int’` directive to encode crucial memory addresses (`GPIO Function Select`, `GPISSET0` and `GPICLR0`) to the head of the executable and "load literal" instruction to load them.

However we found this was not the best approach since our emulator and Raspberry Pi seems handle memory addresses differently. Thus we changed to the wide move instructions.

3.3 The Program

The program first loads all crucial addresses and data as described above. We then set GPIO 2 to output mode by using the `‘str’` instruction to store the pin number to `GPIO Function Select` in the required format: 001000000 in binary; thus 0x0040 in hexadecimal.

Once the pin is configured, we set the LED by writing the pin number to `GPISSET0`: 0010 in binary; thus 0x0002 in hexadecimal. We introduce a fixed time interval by setting a counter and entering a loop, decrementing the counter by 1 each loop and break once it reaches 0 by using the conditional branch instruction. We then clear the LED similarly by writing to `GPICLR0` and wait for another time interval. The program is an infinite loop, so in the end there is an unconditional branch to setting the LED.

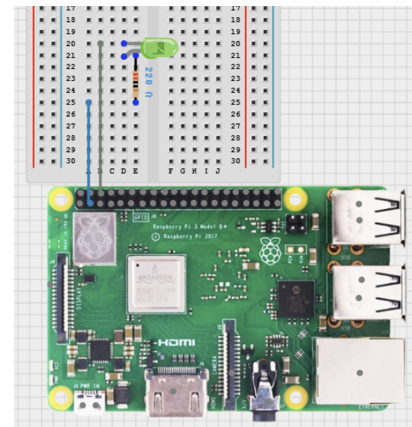


Figure 2: RPI Diagram

4 Project Extension

4.1 Description and Usage

For our C project extension, we have decided to program a vocal synthesiser inspired by Vocaloid. Vocal synthesisers have been used in modern music production, offering a range of applications that **enhance the creative possibilities of sound design**, including experimenting with new sounds, creating backing tracks, etc. This first started off with IBM 7094, singing Daisy Bell, which is also the same song we have decided

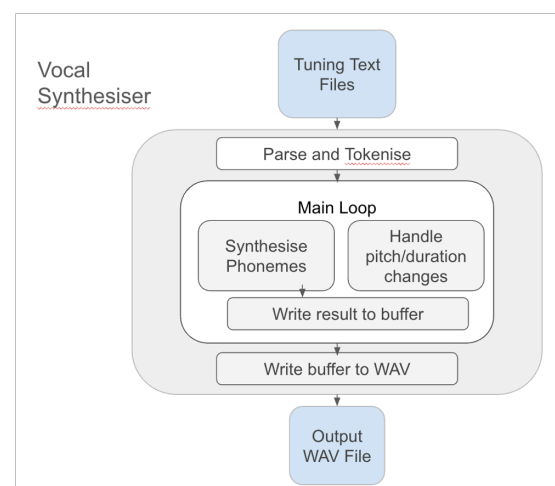


Figure 3: Vocal Synth Structure

to write along with Fairytale by Alexander Rybak and Mockingbird by Eminem.

4.2 Design and Challenges

4.2.1 Definitions

Glottal source: the sound generated by the vibration of the vocal folds as air passes through

Formant frequencies: frequencies which have highest amplitude

Delay lines: memorised inputs and outputs from previous phonemes

Sawtooth wave: non-sinusoidal periodic waveform that rises linearly over each cycle and then drops sharply

Chunk: separates music into smaller more manageable sections

Bandpass Filter: calculates one new audio sample based on the current input and the memory of previous inputs and outputs

4.2.2 Design

Design background For us to speak, air from our lungs push our vocal folds open, while the tension within these folds snap them shut again. This creates a longitudinal wave, and the rate at which the vocal folds open and close is your pitch (also known as f_0).

This wave then travels to your lips. Different shapes formed by the lips, tongue and jaw, have different resonant frequencies. These are called the formant frequencies, where the first three formants cause vowels to sound different to each other [1, 2].

The graph below shows f_0 and subsequent harmonics, with f_1 , f_2 , f_3 being frequencies of greatest amplitude.

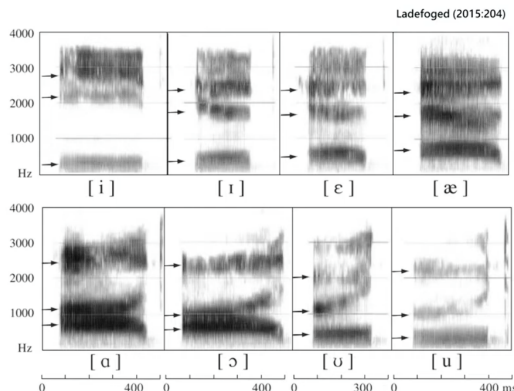


Figure 4: Vowel Formants [3]

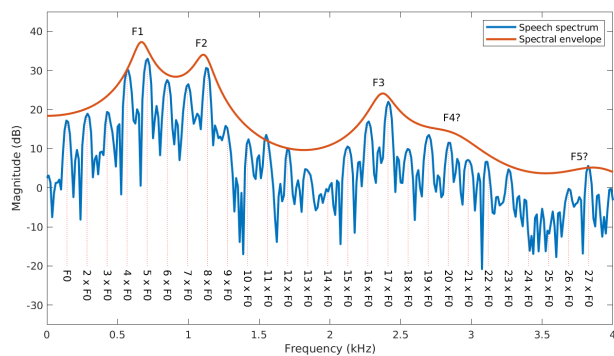


Figure 5: F0 and Formants [4]

Finally, the sound is radiated from the lips into the air.

The source-filter model In vocal synthesis, the glottal source determines the pitch and the filter determines the formants, which encodes which phoneme is being said.

This model allows you to control the source and filter independently, meaning a higher pitched voice singing the same words can simply have a different source, but the same filter.

The entire program put together goes in order of the following: validate input, parse phonemes, run the synthesis loop, and write the output WAV file.

Implementation choices Firstly, to avoid audio clicks, we separated our `init_filter` function from our `update_filter` function, as the latter ensures the delay lines are not reset to 0.

Secondly, our synthesiser should process three formants in parallel. The sawtooth wave is sent into F1-6 in parallel, and the outputs added together, as this is computationally simpler and makes it more scalable, as consonants can use the same model.

The glottal source A sawtooth actually has *too much* high-frequency energy compared to a real human voice, making the generated vocals sound robotic. In real life, human vocal folds do not vibrate in a perfect zigzag. In fact, there are actually periods of silence between each vibration called the Rosenberg pulse, which softens the higher frequencies.

This is why Klatt uses a mathematical model, rather than the sawtooth model. However, for the scope of this project, we are willing to make this tradeoff [5].

Phoneme interpolation When we speak, we glide smoothly between phonemes. If a synthesiser were to jump from the formants of "Ee" directly to the formants of "Ah", the resulting audio would sound robotic and disjointed.

To solve this, we use a `PhonemeFrame` struct that defines the target frequency and bandwidth for F1-6. Between each chunk, the synthesiser interpolates the parameters from the previous vowel target toward the next vowel target. We simulate this using a cosine easing function to make it sound more human.

Once the interpolated formant targets are calculated for a specific chunk, the `synthesize_frame()` function calls `update_filter()` to adjust the Bandpass coefficients without resetting the delay lines, ensuring vowels transition smoothly [6].

However, adding the outputs of F1-6 together equally introduces inaccuracy: the higher frequencies (F3-6) end up artificially loud.

Our solution is to apply a weight to each filter before summing them using their corresponding formant amplitudes derived from human speech. [7].

5 Testing

5.1 Automating the process

Using Makefiles we set up testing and debugging tools like Valgrind and provided test suite and unit test commands to automate the development process. Moreover, we used a CI/CD pipeline to run tests, ensuring each branch passes all the tests before it is merged into master.

5.2 Unit Tests

We tested the pure functions of our assembler via unit tests and ran them through both Ubuntu and Alpine. Through this, we found an error in our parser, as it caused a segmentation fault on alpine, but not ubuntu. We also unit tested the pure functions in the extension to ensure that they were behaving as expected. Floating-point results are checked using absolute comparison, comparing the absolute difference with a tolerance, rather than exact equality, which absorbs rounding variances on different platforms and ensures the tests pass reliably on both Ubuntu and Alpine.

5.3 Evaluation

The biggest challenge of the extension was testing. This is because it is difficult to test audio, as WAV files cannot be simply byte-compared to an expected result. Our solution was to implement a mixture of unit tests and test the audio files by ear. The unit tests were effective in ensuring pure functions returned their corresponding output. Since we do not have access to an expected output, testing the input/output correctness by ear was the most appropriate option because as long as the audio sounds fine, any small inaudible errors would not affect the functionality of the sound. However, this method is not the most accurate method and a lot more manual, meaning hidden bugs may be missed.

A more reliable testing method is analysing the sound wave. We would run an FFT on the output WAV file to extract the peak frequencies and then check that they match the expected formant frequencies. This method would be a more objective approach rather than relying on the human ear, which is very subjective.

6 Group and Individual Reflections

6.1 Group Reflection

Working on the project as a group of 4 required us to split self-contained components between each member, meaning we had to plan our implementation ahead. This proved to have improved as we moved from the emulator to the assembler, as we avoided the need for large refactors that set us back a few days. Communication was primarily done on Discord, where we would let the group know which component we were working on and inquire other group members about implementation strategies before starting a new component. For debugging and part III, we largely worked together in person through pair programming, which proved to speed up the process when it came to debugging, as we were more likely to spot a mistake if we all worked together on the same machine. We decided to remain flexible on how we split the workload, with each member taking a task they would like to implement, and continuing to do so after they completed their part. This worked well, as it meant that if one part took longer than expected, we were not stuck until that component was completed. Overall, in the future, we think establishing a more formal way of splitting the work and letting people know of the progress may be useful, as it would reduce the ambiguity of times where code was not pushed to remote. However, the good communication and mutual code reviews are things we intend to keep for future group projects.

6.2 Individual Reflections

Ashlyn Chor: I was a bit nervous before starting this project, as I had heard about the challenges previous years had to face such as merge conflicts, bad teammates, segmentation faults, etc. This was amplified by the fact that I had never worked on a practical project on a team with git before, nor was I the most familiar with C.

However, as we got into the project, I found myself picking up key git commands and c programming fundamentals fairly quickly and found that I really enjoyed the experience of working with other people. Especially being able to discuss and develop ideas on how to implement something. Although there were some times where I felt communication could have been better within the team. Since due to our different schedules, it is rare to have everyone active at the same time online. But through frequent updates, we were all able to keep in loop and progress the project at a good pace.

For future projects, I think it would be beneficial for us to spend a bit more time before beginning the programming on discussing and finalising the design. This would help us to ensure we all have the same idea on the implementation as well making sure every requirement is fulfilled.

Katie Wan: Before this project, I expected the biggest learning outcomes would be predominantly on the implementation side. However, the peer assessment feedback made me realise that automating the development process via Makefiles and reviewing merge requests were equally important. This tells me that making a short term tradeoff by spending time to properly set up a smooth workflow before starting the implementation is worth the convenience for later development. Initially, I was worried about the difficulty of implementing an unfamiliar topic; however, upon finishing, I enjoyed researching anything I was unsure about, and I am excited to learn more about unfamiliar topics through future projects!

In the future, I think I could improve on planning ahead and defining the problem more thoroughly before starting on the codebase to avoid large refactors. Moreover, I think I could benefit from pair programming more often, as a lot of the time I spent fixing bugs may have been saved if I had asked a fresh pair of eyes to take a look at my implementation.

Suyoung Moon: Through this project, not only have I advanced my coding skills, but I have also improved and learned proper Git conventions and effective development workflows. Communication was another major skill that I have improved during this experience, as facing and resolving unexpected technical challenges as a team significantly improved my collaborative decision-making skills. Looking back to the experience, I realised I could have contributed even more by proactively suggesting new ideas. Personally, I also learned the importance of understanding my role and recognizing the specific value I must bring to a team, as this directly affects the productivity of the team. During the project, I also had the opportunity to learn and investigate the voice synthesiser, a topic that I previously lacked deep knowledge.

Overall, this experience was a valuable step up from my previous background in individual projects. It taught me essential team dynamics to provide more value in future team environments.

Jiahui Yan: I primarily worked alone in the past. The most significant difference in the group project is that there is no longer a single person who make decisions. I thought I would use Git well - since I have used it for a while in solo projects, but I soon realised that Git in teamwork is very different. Also, I used to work in my own pace - that is not quite fast - in the past. This project really let me know how seeing my teammates acting fast can also stimulate myself to working faster and getting over laziness.

In the project I worked in conceptual things, such as an error handling framework and a parser for the vocal synthesiser. I also did general implementations for other parts and chores like unit testing and documenting the codebase.

Sometimes I still feel that I am not integrating in the group very well by getting confused about what my teammates are doing or making duplicate changes. This is definitely something I can improve in the future.

References

- [1] ScienceInsights. What are formants and how do they shape speech? [Internet]. ScienceInsights; 2025 Nov 18 [cited 2026 Jun 16]. Available from: <https://scienceinsights.org/what-are-formants-and-how-do-they-shape-speech/>

- [2] Formant. In: Wikipedia [Internet]. San Francisco (CA): Wikimedia Foundation; [cited 2026 Jun 16]. Available from: <https://en.wikipedia.org/wiki/Formant>
- [3] EdUHK. 2.2. Formants of Vowels – Phonetics and Phonology [Internet]. Eduhk. 2025. Available from: https://corpus.eduhk.hk/english_pronunciation/index.php/2-2-formants-of-vowels/
- [4] Fundamental frequency (F0) — Introduction to Speech Processing [Internet]. speechprocessingbook.aalto.fi. Available from: https://speechprocessingbook.aalto.fi/Representations/Fundamental_frequency_F0.html
- [5] Klatt DH. Software for a cascade/parallel formant synthesizer. J Acoust Soc Am. 1980;67(3):971-95.
- [6] Bristow-Johnson R. Cookbook formulae for audio EQ biquad filter coefficients [Internet]. 1994 [cited 2026 Jun 16]. Available from: <https://webaudio.github.io/Audio-EQ-Cookbook/audio-eq-cookbook.html>
- [7] Gies T. klattsch: Primitive parallel-formant speech synth in the browser (Version 0.5.0) [Software]. GitHub; 2026 [cited 2026 Jun 16]. Available from: <https://github.com/tgies/klattsch>